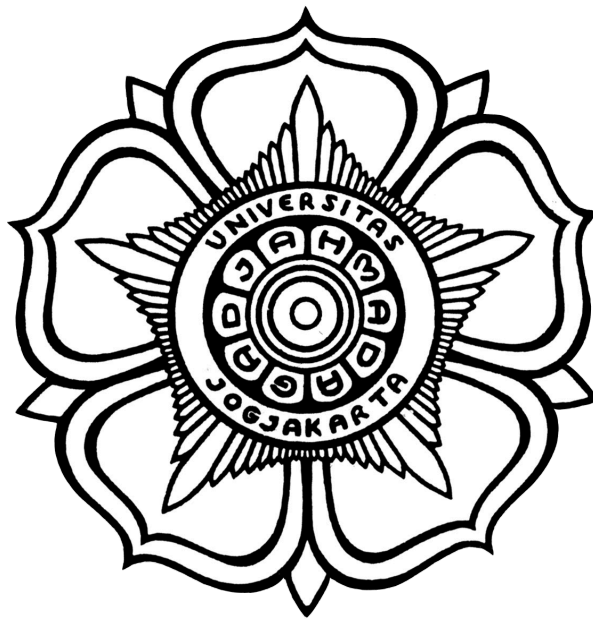


Week 10 - Scalable Software Design

Design Your Own Scalable Software

10-05-2026



Team 5: Foodiction

Yohana Butar Butar - 24/546690/PA/23212

Aufaa Azzahra Aryawan - 24/546854/PA/23215

Ratu Faiha Salsabilla Rahmadina - 24/532756/PA/22533

Regina Joan Medea Jati Laksono - 24/532850/PA/22546

Section 1 - System Overview and Requirement

Foodiction is an intelligent local food discovery platform designed to digitize and preserve Yogyakarta's rich culinary heritage. Beyond simple search, the system utilizes a data driven approach to match user profiles with real time variables like peak hours and trending locations. This ensures that recommendations are not only relevant but also dynamic, allowing the system to scale its intelligence as more data is processed, ultimately creating a robust ecosystem for local food tourism.

By integrating factors such as the user's cravings, budget constraints, and current time of day, the platform transforms a routine meal choice into a curated cultural experience. Whether a traveler is seeking a hidden Angkringan gem at midnight or a student is looking for an affordable Gudeg for breakfast, Foodiction removes the choice paralysis often found in traditional search engines. This personalized touch not only enhances user satisfaction but also provides vital digital visibility to small-scale local vendors, fostering a sustainable connection between the city's gastronomic traditions and the modern digital consumer.

Functional Requirements

- User Preference Matching: Personalized filtering based on taste (spicy, savory, salty), distance, and budget ranges.
- Time Recommendations: Automated suggestions based on the time of day (breakfast vs. late night snacks).
- Localized Discovery: A dedicated category for Yogyakarta specific food categories like Gudeg, Bakmi Jawa, and Angkringan.
- Social Proof Review: A rating and review feature to ensure recommendation quality and user engagement.

Non-Functional Requirements

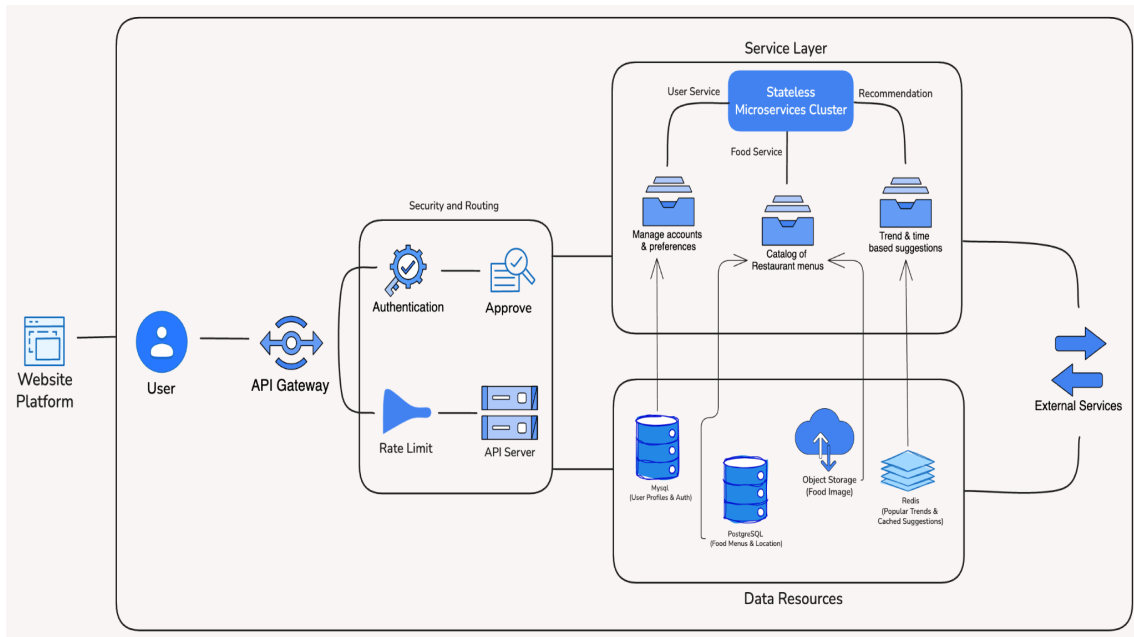
- Availability: Target 99.5% uptime to ensure reliability for users on the go.
- Latency: Response time for recommendations must be under 2 seconds.
- Scalability: The system must handle sudden traffic spikes during holiday seasons especially in high concurrency.
- Statelessness: The application logic must be stateless to allow seamless container scaling.

Traffic Estimate

- Normal Load: 10 - 50 Requests Per Second (RPS).
- Peak Load: Up to 250 RPS (during lunch/dinner hours or long weekends).

Section 2 - Architecture Design

For our project, we have chosen a Microservices approach over a Monolith. This allows independent scaling of the Recommendation Engine, which is computationally heavier than the User Profile service. It also ensures that a failure in the review service does not bring down the entire food discovery feature in the system.



Component Boundaries & Responsibilities

- **Website Platform:** A responsive interface for users to interact with the system.
- **API Gateway (Nginx/Kong):** The single entry point that handles request routing, authentication, and rate limiting.
- **User Service:** Manages user accounts and preferences (Relational DB: MySQL).
- **Food Service:** Manages the catalog of local eateries and menus (Relational DB: PostgreSQL).
- **Recommendation Engine:** A dedicated service that processes rules and trends to generate suggestions.
- **Storage:** External Object Storage for hosting high quality food images.

To ensure the system is Docker ready, we will apply:

- **Base Images:** We will use lightweight images like `node:alpine` or `python:3.9-slim` to keep the container footprint small and deployment fast.
- **Inter Service Communication:** Services will communicate via RESTful APIs over a private Docker network.

- **Stateless Design:** No data will be stored inside the container's writable layer. All persistent data is offloaded to managed database instances or external volumes.
- **Environment Parity:** Using docker-compose, we ensure that the development environment is an exact replica of the production environment.

Issue: Standard file uploads (like restaurant photos) usually break in containers when the container restarts.

Solution: We address this by using Centralized Storage. Instead of saving files to local/path, the application will stream uploads directly to an S3 compatible bucket, ensuring all container instances see the same data.

Section 3 - Scaling Strategy

Foodiction is designed to support increasing numbers of users, restaurants, reviews, and recommendation requests without significant performance degradation. Since food recommendation traffic can fluctuate heavily during lunch hours, dinner time, holidays, and weekends, the system adopts both horizontal and vertical scaling strategies depending on the characteristics of each component.

Horizontal and Vertical Scaling

The system mainly scales horizontally at the application layer. Stateless services such as the API gateway, User service, Food service, and Recommendation engine can run multiple container instances simultaneously behind a load balancer. This allows incoming traffic to be distributed evenly across containers and prevents a single server from becoming overloaded.

The recommendation engine is the most computational intensive component because it processes user preferences, trending food data, time-based conditions, and restaurant ratings to generate personalized recommendations. Therefore, this service is designed to scale horizontally first whenever recommendation requests have a sudden increase. For example, during dinner hours in Yogyakarta, many users may simultaneously search for 'cheap spicy chicken near UGM'. In this instance, additional recommendation engine containers can automatically be deployed to maintain low response times.

Meanwhile, stateful infrastructure such as MySQL and PostgreSQL databases scale vertically by increasing CPU, memory, and storage resources. Database systems benefit more from stronger hardware because relational queries, indexing, and transactional consistency are resource intensive and more difficult to distribute across many nodes compared to stateless application services. Furthermore, Redis caching infrastructure may also scale vertically because larger memory capacity directly improves cache hit rates and recommendation retrieval speed.

Statelessness Strategy

To support container scalability and fault tolerance, Foodiction follows a stateless service design wherever possible. Stateless services do not store session information or persistent application data inside the container itself. Instead, all persistent data is stored externally in databases, object storage, or caching systems.

The following services are designed to be stateless:

- API gateway
- User service
- Food service
- Recommendation engine

User authentication is handled using token based authentication such as JWT, allowing any service instance to process incoming requests without relying on local session storage.

Persistent user information, reviews, menus, and recommendation metadata are stored in external databases, while uploaded restaurant images are stored in centralized object storage. This ensures containers can be started, stopped, or replaced without affecting application data.

This design improves:

- Scalability
- Reliability
- Fault recovery
- Deployment flexibility in Docker based environments

Auto-Scaling Strategy

Foodiction supports automatic scaling using container orchestration platforms like Kubernetes in future deployment environments. The system uses performance metrics to determine when additional service containers should be deployed.

Scale-Out Triggers

New container instances are created when:

- CPU utilization > 70%
- Memory usage > 75%
- Recommendation response latency exceeds acceptable thresholds
- Request volume significantly increases during peak meal hours

For example, recommendation traffic may increase sharply during lunch and dinner hours; especially during long weekends or tourist seasons in an area (like Yogyakarta). Under these circumstances, the orchestration platform automatically deploys additional recommendation engine containers.

Scale-In Conditions

Container instances are reduced when:

- CPU usage remains below 30%
- Request traffic decreases for an extended period
- Response latency returns to normal levels

This prevents unnecessary infrastructure costs during off-peak periods whilst maintaining system responsiveness during high traffic conditions.

Scalability Goals

The system is designed to scale progressively as Foodiction expands to support:

- Larger user populations
- More restaurants and menu data
- Increased review activity
- Additional cities outside of Yogyakarta

Since recommendation quality improves as more interaction data becomes available, the architecture supports long term growth without requiring major redesigns. In addition, the microservices approach allows new services and recommendation features to be introduced independently without disrupting the overall system.

Section 4 - Data Layer Design

Foodiction uses a hybrid data storage architecture that combines relational databases, caching systems, and external object storage to balance consistency, scalability, and performance.

SQL Database Selection

The system primarily uses relational databases because most of the application data contains structured relationships that require transaction consistency.

MySQL - User service

The User service uses MySQL to manage:

- User accounts
- Authentication data

- Saved preferences
- Review history

MySQL is suitable because user related data requires strong consistency and reliable relational queries.

PostgreSQL – Food service

The Food service uses PostgreSQL to store:

- Restaurant information
- Food menus
- Food categories
- Ratings
- Geographic location data

PostgreSQL was chosen for our system because of its strong support for complex queries and structured relational data. This is crucial when filtering recommendations based on food categories, pricing, distance, and restaurant popularity. Moreover, relational databases ensure that relationships between users, restaurants, menus, and reviews remain consistent and accurate.

Recommendation Data and Trend Processing

The recommendation engine processes user behaviour, trending searches, and recommendation scores generated from user interactions. Since recommendation related data changes frequently and grows rapidly, some temporary recommendation metadata may be stored in Redis for a faster retrieval and a reduced database load.

Some examples include:

- Trending food searches
- Frequently accessed recommendations
- Popular restaurants
- Time sensitive recommendation results

This allows the system to quickly return recommendation results without repeatedly querying the relational databases.

Caching Strategy

Foodiction uses Redis as an in-memory caching layer to improve response speed and reduce database traffic. The following data below are the cached content:

- Popular recommendation results
- Frequently searched food categories

- Trending restaurants
- Nearby food suggestions
- Restaurant menu data

For example, if many users search for ‘late night Gudeg near UGM’, the recommendation results can be temporarily stored in Redis so that future requests can be given immediately without recalculating recommendations.

Cache Eviction and Invalidation

The caching layer uses:

- Time To Live (TTL) expiration
- Least Recently Used (LRU) eviction policies

Recommendation caches automatically expire after a short duration to ensure that users receive updated restaurant ratings and trending food information. Cache invalidation occurs when:

- Restaurant menus are updated
- Ratings significantly change
- Recommendation calculations are refreshed

This prevents stale or outdated recommendations from being displayed to users.

Replication and Data Availability

To improve reliability and read performance, the relational databases can use a primary replica topology.

- Primary databases handle write operations
- Replica databases handle read heavy operations like recommendation queries and restaurant searches

This reduces pressure on the primary database whilst improving scalability for concurrent users. Additionally, database backup and replication improve fault tolerance and system availability in case of infrastructure failure.

Object Storage Strategy

Food images and restaurant media files are not stored inside application containers because container local storage is temporary and unsuitable for scalable environments. Instead, Foodiction uses centralized S3 compatible object storage for:

- Restaurant photos
- Menu images
- Promotional content

This ensures all container instances can access the same media assets consistently while supporting scalable deployments across multiple servers.

Consistency Trade-Offs

Foodiction prioritises strong consistency for critical transactional data such as:

- User accounts
- Restaurant information
- Restaurant ratings

However, recommendation trends and cached popularity scores may use eventual consistency because small delays in updating recommendation rankings do not significantly affect user experience. This trade off improves scalability and system responsiveness whilst maintaining accuracy for essential application data.

Section 5 - Load Balancing and CDN

Load Balancer Placement in the Architecture

Foodiction employs a two tier load balancing strategy that aligns with the microservices architecture defined in Section 2. The first tier is an external load balancer positioned at the network edge, directly facing incoming client traffic from the internet. This load balancer sits in front of the API Gateway layer, which is implemented using Nginx or Kong. Its primary role is to distribute HTTP and HTTPS requests across multiple API Gateway container instances, thereby eliminating the API Gateway as a single point of failure.

The second tier operates internally within the Docker private network. Stateless services such as the User Service, Food Service, and Recommendation Engine are deployed as multiple container instances behind internal load balancing mechanisms. In the current Docker Compose development environment, Docker's embedded DNS resolver performs rudimentary load distribution by resolving service names, such as recommendation engine, to the IP addresses of available containers. In future Kubernetes deployments, this internal distribution will be managed by Kubernetes Service resources, which expose a stable virtual IP that routes traffic to healthy pods regardless of container lifecycle events.

This two tier placement ensures that external traffic is first normalized and secured at the edge, while inter service communication remains resilient and abstracted from individual container instances.

Load Balancing Algorithm Choice and Justification

For the external tier, Foodiction utilizes a hybrid algorithm approach. Under normal operating conditions, defined in Section 1 as 10 to 50 Requests Per Second, the system employs Round Robin distribution. Round Robin rotates incoming requests sequentially across all healthy API Gateway instances. This algorithm is stateless, predictable, and trivial to implement, which aligns with Foodiction's stateless design requirement.

During peak load conditions, specifically when traffic approaches or exceeds 250 Requests Per Second during lunch hours, dinner hours, or long weekends in Yogyakarta, the algorithm shifts to Least Connections. Least Connections directs each new request to the instance with the fewest active connections at that moment. This is justified because recommendation queries and image heavy discovery pages exhibit variable processing times. An instance handling a computationally expensive recommendation request should not receive additional connections while other instances remain idle. Least Connections prevents request backlog on slow instances and supports the sub two second latency target specified in Section 1.

For the internal tier, Docker's default routing mechanism operates as a basic round robin resolver. This is sufficient because internal REST API calls between microservices are lightweight and uniform in duration compared to external user facing requests.

Reverse Proxy Configuration

The API Gateway functions as a reverse proxy for all downstream microservices. In this capacity, Nginx or Kong terminates incoming TLS connections, applies rate limiting to prevent abuse, and routes requests to the appropriate backend service based on URL path prefixes.

The reverse proxy configuration defines upstream blocks for each service cluster. For example, the Recommendation Engine upstream block lists the addresses of all active container instances. The proxy passes requests to these upstreams using the selected load balancing algorithm. Additionally, the reverse proxy strips internal headers, adds request tracing identifiers for observability, and enforces timeout policies. If a downstream service does not respond within the configured threshold, the reverse proxy returns a standardized error response rather than hanging indefinitely, which preserves user experience during partial outages.

CDN Integration

Foodiction integrates a Content Delivery Network to serve static assets, specifically the high resolution restaurant photographs and menu images stored in the S3 compatible object storage described in Section 4. These assets are large, read heavy, and geographically distributed to users across Yogyakarta and surrounding regions. Serving

them directly from object storage origin would introduce latency, increase egress costs, and consume bandwidth that could otherwise support dynamic API traffic.

Assets Served via CDN:

- Restaurant profile photographs
- Menu item images
- Promotional banners and thumbnails

Cache Policy:

The CDN applies a time based caching policy with a Time To Live of 24 hours for standard menu images and 7 days for promotional content that changes infrequently. When a restaurant uploads a new image, the Food Service triggers a cache invalidation event, detailed in Section 6, which purges the specific CDN edge cache for that asset. This ensures that menu updates are reflected to users without waiting for natural TTL expiration.

For assets that have not been modified, the CDN serves subsequent requests from the geographically nearest edge server. This reduces image load latency from approximately 150 milliseconds when fetched from origin storage to approximately 20 milliseconds from edge cache, directly supporting the latency requirements of Section 1.

Section 6 - Asynchronous and Event-Driven Components

Workflow Identification: User Review Submission

The workflow selected for asynchronous processing is the user review submission and downstream rating update cycle. When a user submits a review for a restaurant, the system must persist the review, recalculate the restaurant's average rating, update the restaurant's trending score within the Recommendation Engine, and invalidate the relevant Redis cache entries.

Why Synchronous Processing Is Insufficient

A purely synchronous implementation would require the User Service to execute the following sequence within a single request response cycle: write the review to MySQL, invoke the Food Service to recalculate the average rating, invoke the Recommendation Engine to recompute the trending score, and finally invalidate the Redis cache. This approach is insufficient for three reasons.

First, it violates the sub two second latency requirement. During peak hours, the Recommendation Engine may be under heavy computational load. Forcing the user to wait for cascading calculations would result in response times exceeding five seconds.

Second, it creates tight coupling between services. A temporary failure or slowdown in the Recommendation Engine would cause the entire review submission to fail, even though the primary objective, persisting the review, has already succeeded.

Third, it undermines the stateless fault tolerance goals of Section 3. Synchronous chains amplify failure domains. Asynchronous decoupling ensures that the critical user facing path remains fast and reliable, while secondary computations proceed in the background.

Producer Consumer Topology Design

Foodiction implements a publisher subscriber topology using the existing Redis infrastructure already deployed for caching in Section 4. This avoids the operational overhead of introducing a separate message broker.

Producer: The User Service acts as the event producer. Immediately after persisting a review to MySQL, it publishes a JSON event to the Redis channel `events:review_created`.

Consumers: Three independent subscriber services consume from this channel:

1. **Food Service Consumer:** Recalculates the restaurant's average rating and review count in PostgreSQL.
2. **Recommendation Engine Consumer:** Updates the restaurant's trending score and regenerates recommendation metadata.
3. **Cache Invalidation Consumer:** Deletes the Redis keys associated with the affected restaurant's cached details and recommendation results.

Each consumer maintains its own subscription to the channel. Redis Pub/Sub broadcasts the event to all subscribers simultaneously, allowing parallel processing. The consumers operate as background worker processes within their respective service containers.

Backpressure Handling

Backpressure occurs when consumers process events more slowly than the producer generates them, causing memory or latency issues. Foodiction addresses this through the following mechanisms.

- **Consumer Scaling:** Because the subscriber consumers are stateless, additional consumer instances can be deployed horizontally during peak review activity. For example, if a viral social media post drives hundreds of simultaneous reviews to a single trending restaurant, the orchestration layer can spawn additional Cache Invalidation Consumer containers to drain the event backlog.
- **Idempotency:** Each event carries a unique `review_id`. Consumers are designed to be idempotent. If an event is processed multiple times due to retry logic or

duplicate delivery, the database state remains correct because the consumer checks for the existence of the review ID before applying updates.

- **Monitoring and Alerting:** Consumer lag is monitored by tracking the age of the oldest unprocessed event reference. If lag exceeds a configurable threshold, for example 30 seconds, the system triggers an alert and auto scaling policy to deploy additional consumer capacity.
- **Graceful Degradation:** If the Recommendation Engine consumer falls behind, the system continues to serve recommendations using slightly stale cached data. This is acceptable because recommendation trends, as noted in Section 4, operate under eventual consistency. The user experience degrades gracefully rather than failing outright.

Consistency Trade Offs Introduced by Async Processing

The shift to asynchronous processing introduces a deliberate consistency trade off between the review database and the derived data stores.

Strong Consistency Domain: The review data itself, stored in MySQL by the User Service, remains strongly consistent. Once the user receives the confirmation response, the review is durably persisted. This satisfies the transactional integrity requirement for user generated content.

Eventual Consistency Domain: The average rating in PostgreSQL, the trending scores in the Recommendation Engine, and the Redis cache are eventually consistent. In the interval between review persistence and full consumer processing, which typically lasts less than one second but may extend during peak load, a user may observe the following anomalies:

- The restaurant page displays the new review but an outdated average rating.
- A recommendation list excludes the newly reviewed restaurant from trending rankings until the Recommendation Engine consumer completes its update.

This trade off is justified because the anomalies are transient, user visible only under narrow timing conditions, and far preferable to the latency and reliability failures of a fully synchronous model. As stated in Section 4, Foodiction explicitly prioritizes strong consistency for transactional data while accepting eventual consistency for derived analytical data such as trends and cached popularity scores.

Section 7 - Bottleneck Analysis and Trade-o Discussion

Likely Bottlenecks

The Recommendation Engine is the part of Foodiction most likely to become a problem

at scale. While other services mostly read and write to their respective databases, the engine has to process preferences, trends, location, and timing in a single request cycle. Based on Amdahl's Law, the system's throughput ceiling is set by the computation that cannot be distributed, particularly the Redis reads that require coordination across instances. Horizontal scaling of the engine helps but hits diminishing returns sooner than the other services.

The database write path is the second weak point. Since both MySQL and PostgreSQL scale vertically, high volumes of concurrent review submissions, as described in Section 6, all converge on one primary node per database. This creates a queue that read replicas simply are not designed to solve.

What the Design Sacrificed

Eventual consistency was accepted for all derived data, meaning ratings, trending scores, and recommendation caches will lag slightly behind the actual state of the database after a review is submitted. The tradeoff is reasonable given the use case, but it is a real limitation. Microservices were chosen over a monolith knowing that it would increase complexity for a small team. The benefit is meaningful though, since the Recommendation Engine being independently deployable is core to the scaling strategy. Redis Pub/Sub was used in place of a proper message broker to avoid extra infrastructure, but this creates a gap where a Redis restart during peak load results in permanent event loss with no automatic recovery.

Single Points of Failure and Mitigation

The external load balancer has no failover mechanism in the current design, making it a quiet but significant risk. Redis carrying both the cache and the event channel means a crash has compounding effects across recommendation quality and review processing. The biggest operational gap is still the database primary nodes: a failure in either MySQL or PostgreSQL halts all writes until a replica is manually promoted. Redis Sentinel and Patroni are the two tools that most directly address these risks and would be the first additions in a production-ready version of the system.

Future Iteration Priorities

The most actionable next steps coming out of this analysis are adopting a durable message broker to replace Redis Pub/Sub, implementing automated failover for the database layer, and investigating whether the Recommendation Engine's heavier computations can be moved to off-peak pre-processing. These improvements address the core weaknesses identified above and can be applied incrementally without overhauling the existing design.